

Database access objects (DAO)

- These are the main components of Room because they are responsible for defining methods that access the database.
- They define the methods for interacting with the database.
- The implementation is auto-generated at compile-time (annotation processing).

Database

- This defines the database holder.
- It is the main access point to the database connection.
- It defines the list of entities and database access objects (DAO).

Let's assume our news app stores the articles in the SQLite database so that users can read the news even if offline. The entity will represent news articles as shown below:

```
@Entity(tableName = "news_articles")
data class NewsArticle(
    @PrimaryKey(autoGenerate = true)
    var id: Int = 0,
    var author: String? = null,
    var title: String = "",
    var description: String = "")
```

This will create a table called 'news_articles' in the SQLite database of the app, and use 'ID' as the primary key. To access the 'news_articles' table, we will need to create a DAO (database access object) to provide abstraction over the database logic, and provide a way to interact with the underlying database.

To define a DAO, we need to create an interface and annotate it with `@Dao` annotation, and define the methods to play with the 'news_articles' table.

The DAO for 'NewsArticle' will look something like what's shown below:

```
-
@Dao
interface NewsArticlesDao {

}
```

Insert

Inserting a record is pretty easy. We just need to add a function in the DAO and annotate it with the `@Insert` annotation.

```
@Dao
interface NewsArticlesDao {

    @Insert
    fun insertArticle(article: NewsArticle)

}
```

You can even bulk-insert lists or arrays of 'NewsArticle' as shown below:

```
@Insert
fun insertArticles(articles: List<NewsArticle>)
```

Update

Similar to *insert*, to *update* a query, we just need to annotate the function with `@Update` annotation.

```
@Update
fun updateArticle(article: NewsArticle)
```

Internally, the Room library performs the *update* operation using the primary key of the entity.

Delete

To *delete* a query, we just need to annotate the function with `@Delete` annotation.

```
@Delete
fun deleteArticle(article: NewsArticle)
```

Internally, the Room library performs the *delete* operation using the primary key of the entity.

Read/select

Select queries are a little different with Room. We need to write the SQL query, which is verified at compile time (you also get Autocomplete, which is awesome). To perform *select* queries, we need to use the `@Query` annotation and specify the SQL query; then Room internally maps the query result with the Kotlin/Java class.

```
@Query("SELECT * FROM news_articles")
fun getNewsArticles(): List<NewsArticle>
```

You can even get observable data using LiveData, as follows:

```
@Query("SELECT * FROM news_articles")
fun getNewsArticles(): LiveData<List<NewsArticle>>
```

This way, you'll be able to update the UI when something in the table changes.

On the other hand, if you are a fan of RxJava, you can also get *Maybe*, *Single*, *Flowable* instead of LiveData.

```
@Query("SELECT * FROM news_articles")
fun getNewsArticles(): Flowable<List<NewsArticle>>
```

Benefits of Architecture Components

Some of the main advantages of Android Architecture Components are given below.